

Email Phishing Detection with an Emphasis on Adversarial Robustness and Explainability

Lia Bercovitz

Data Science & Cyber – the content layer of a two-part (fused) phishing detector

Abstract

Phishing email is still a leading entry point for attacks, and a lot of the signal lives in the message *text*. In this project I build the content layer of a larger, fused phishing detector: this layer reads only the subject and body, while a second, independent layer (a partner detector) reads the URLs and sender metadata, and the two are combined into one system. I study my layer along three axes: detection quality, explainability, and - the main focus - robustness to text-obfuscation evasion. I use a classical text-classification pipeline (TF-IDF with Naive Bayes / Logistic Regression). On CEAS_08, the one large corpus with both classes from a single source, the task is easy on clean text (Logistic Regression F1 0.998); I treat that near-perfect score as a property of clean, single-corpus data, not a claim about the wild, and the rest of the work is a deliberate arms race. A word-only model is fragile: an imperceptible homoglyph attack drops recall from 0.99 to **0.71**. Character n-grams and a Unicode-normalisation defence restore it and even repair an *unseen* full-width attack, but an adaptive, importance-guided attacker opens a residual gap - which I then strongly reduce by folding through the full Unicode confusables set (UTS #39), the official list browsers use, so the same obfuscation is much harder to land and recall returns to near its clean value. Honest evaluation shows the laboratory number survives neither a change of corpus (F1 0.998 → **0.37**) nor time (a 2008 model catches **14%** of real 2023–2025 phishing); *evolving* on modern mail lifts that to 0.98 with no forgetting. A word-level *dilution* attack still evades the model (→ **0.13**), answered by a monotone (non-negative-weight) model whose weights can only add suspicion, so inserted text cannot un-flag a scam, at an honest cost of clean F1 0.998 → **0.912**. Beyond the core detector I also sketch several forward-looking ideas from the presentation: an LLM-paraphrase attack (one LLM rewrites the scam, a second judges it is still phishing), a hand-crafted sentiment/urgency feature, edit-distance (Levenshtein) matching against known brand words, and a self-updating attack database that retrains on newly confirmed attacks. Every headline number carries a bootstrap confidence interval, and classifiers are compared with McNemar’s test. Where a defence is incomplete I say so - which is the core argument for keeping this content layer as one part of a fused system with an independent (URL/sender) detector.

Contents

1	Introduction	2
2	The Pipeline, Step by Step	2
3	Background	3
4	Methodology	4
5	Clean Detection and Exploratory Analysis	5

6	Adversarial Robustness - the Main Result	6
7	Reality Checks: Space and Time	8
8	The Next Attack Class: Words, not Letters	8
9	Keeping It Working, and Deployment	9
10	Discussion	10
11	Conclusion	11

1 Introduction

Phishing is a social-engineering attack in which a message impersonates a trustworthy sender to trick the recipient into revealing credentials, transferring money, or running malware. It is a top initial-access vector in real breaches, and email is its main channel. Because so much of the deception is carried by *language* - urgency (“your account will be suspended”), authority (“security team”), reward, and calls to action (“verify now”) - reading the text is a natural way to catch it.

This project is not meant to stand alone. It is the *content layer* of a larger, more complete phishing detector that fuses two independent signals: my layer reads only the subject and body and decides phishing vs. legitimate, and a partner layer reads the URLs and sender metadata. The two are later fused into one decision, so that when an attacker defeats one channel the other still fires. To keep the split clean I deliberately strip URLs from the text (Section 4) - a content model should earn its score from language, not links. Two properties beyond raw accuracy drive the work. First, **explainability**: an analyst needs to know *why* a message was flagged. Second, and most importantly, **robustness**: a favourite evasion trick is to mangle the text - *vérify* with a look-alike character, *c11ck* in leetspeak - so a human still reads the word but a keyword filter does not. A detector that is excellent on clean mail but fragile on trivially obfuscated mail is not much use.

Contributions. This is an empirical study, and its value is rigour and honesty. (1) A confound-controlled measurement of lexical detection on a single mixed-class corpus, with a robustness study that separates *seen*, *held-out*, and *adaptive* attacks so a defence is never tested against its own inverse. (2) An *arms race* in which each defence provokes a smarter attack, ending with a principled UTS #39 canonicaliser (the official Unicode confusables list) that greatly hardens the character front. (3) Honest external validity: the in-corpus number survives neither a change of corpus nor time, and *evolving* on modern mail recovers it. (4) A second attack class - word-level *dilution* - answered by a monotone model whose weights only ever add suspicion, with its accuracy cost stated plainly. I replicate two classical results (Complement NB under imbalance [Rennie et al., 2003]; lexical fragility to visual attacks [Eger et al., 2019]), report bootstrap confidence intervals and McNemar tests, and state every incomplete defence as such.

2 The Pipeline, Step by Step

Before the detail, here is the whole story in order - how the detector actually evolved, and why each piece exists. Every step below is a real experiment in the sections that follow; this is the map.

1. **Clean the data and remove the easy shortcut.** Merge the corpora, strip URLs (the partner’s job), and rebalance to a realistic legit-majority inbox. Crucially, prove that a model could otherwise cheat by guessing *which corpus* a mail came from, and remove that shortcut by working inside one mixed-class corpus. Freeze and hash the snapshot so every number reproduces.
2. **Build a strong, explainable baseline.** TF-IDF features (word + character n-grams) into Logistic Regression. On clean in-corpus mail this is near-perfect and fully interpretable.
3. **Attack it.** Apply an imperceptible homoglyph/leetspeak attack. The word-only model collapses - a third of phishing gets through with no visible change to a reader.
4. **Fix it.** Add character n-grams and a Unicode-normalisation cleaner. Recall is restored, and it even repairs an attack family I never trained on.
5. **Attack the fix.** Let the attacker be adaptive and target the highest-weight tokens, using look-alike letters deliberately absent from my table. A residual gap reopens.
6. **Strengthen the cleaner.** Fold every character through the full Unicode confusables set (UTS #39), the official list browsers use for spoofed domains. Recall returns to near its clean value and the same obfuscation becomes much harder to land - a big improvement, not just a patch.
7. **Check reality.** Test across corpora and across time. The laboratory number does not survive either: a 2008 model catches only 14% of real 2023–2025 phishing.
8. **Evolve.** Retrain on modern mail. Recall on modern phishing jumps from 0.15 to 0.98 with no forgetting - and a single reported example per campaign recovers most of the rest.
9. **Find the next attack class.** With letters closed, the attacker edits *words*: a dilution attack pads the mail with benign text and evades the model.
10. **Close that too, honestly.** Train a monotone (non-negative-weight) model whose weights can only add suspicion, so inserted text cannot un-flag a scam - and I state its accuracy cost plainly rather than hiding it.

The recurring lesson runs through all ten steps: every component I add, including the ones meant to help, becomes something an attacker can aim at - which is exactly why this layer is built to sit inside a fused system, not to stand alone.

3 Background

The statistical approach to junk mail goes back to Sahami et al. [1998], who framed spam filtering as Naive Bayes text classification; a recent review of language-based phishing detection is Salloum et al. [2022]. These bag-of-words methods are fast and interpretable but blind to word order and brittle to obfuscation. Character n-grams have long given more robust text categorisation [Cavnar and Trenkle, 1994], which I exploit: `cl1ck` still shares letter-chunks with `click`. The idea that small, human-imperceptible perturbations can flip a classifier comes from the adversarial-examples literature [Goodfellow et al., 2015]; in text, character-level attacks such as DeepWordBug [Gao et al., 2018] and TextBugger [Li et al., 2019] insert typos and look-alikes while keeping text readable, and Boucher et al. [2022] show imperceptible Unicode tricks break deployed text systems. Eger et al. [2019] study *visual* attacks and shielding, and Pruthi et al. [2019] defend by normalising text before classification - the strategy I follow, though I use a rule-based normaliser derived from the Unicode

security data [The Unicode Consortium, 2023]. A separate, older evasion strategy *adds* innocuous content: Lowd and Meek [2005] formalised the *good-word attack*, my dilution attack (Section 8) is an instance, and Fleshman et al. [2018] show (for malware) that constraining a model to non-negative weights makes added features only *increase* the maliciousness score - a construction I adopt in Section 8. I use DistilBERT [Sanh et al., 2019] as a transformer baseline and report Matthews Correlation Coefficient throughout, which Chicco and Jurman [2020] argue is more informative than F1 or accuracy on imbalanced problems.

4 Methodology

Data and the corpus-confound problem. I use the Kaggle “Phishing Email Dataset” compilation, which ships several classic corpora as labelled CSVs. A subtle danger is *corpus confounding*: if all phishing came from one corpus and all legitimate mail from another, a classifier could score highly just by recognising *which corpus* a message came from. I demonstrate this directly - a classifier predicting a phishing email’s source corpus reaches **98.8%** accuracy against a 33% baseline - so all main experiments use a single corpus, CEAS_08, the only large corpus containing *both* classes from the same source; the confound is then impossible by construction. Two deliberate choices follow. **(1)** URLs are stripped and replaced by a URL token, so the content model cannot ride on the link signal that belongs to the partner layer. **(2)** I rebalance to a legit-majority base rate (35% phishing): real inboxes are mostly legitimate, and a phishing-majority prior actually *hides* evasion (a model that loses its signal falls back to the majority class). The working set has 26,633 emails; a SHA-256 hash of the frozen snapshot is stored beside the results, so the exact data and every number reproduce (with a fixed seed the whole pipeline is deterministic).

The datasets, and how I cleaned them. The main set is CEAS_08 (26,633 emails, both classes). Four more corpora are held for fairness and reality checks only, never for training the main model: NAZARIO/ENRON/LING (cross-corpus), SPAMASSASSIN (5,809, a second re-check), 1,303 real **2023–2025** phishing emails, and 3,000 recent (2024) legitimate emails. Cleaning is identical on every run:

- strip web links and replace each with a URL token - the link signal is the partner layer’s job, not mine;
- tidy whitespace and cap over-long emails, so a few huge messages cannot dominate;
- drop exact duplicates *before* the split, so no email can leak between train and test;
- rebalance to a realistic 35%-phishing inbox (a legit-majority prior);
- split stratified 60/20/20 with a fixed seed, fitting the vectoriser on training data only;
- hash the frozen snapshot (SHA-256) so every reported number reproduces exactly.

Two quick data checks back the choices: phishing emails are much shorter than legitimate ones (the EDA below), and a date sanity-check found 0 of 34,342 dates hitting the Unix-epoch “1970” trap.

Features and models. Each email (subject + body) becomes a **TF-IDF** vector: term frequency counts a term’s occurrences, inverse document frequency up-weights rare discriminative ones such as “verify account”. I compare **word** (1–2 grams), **char** (3–5 grams), and their **union**, because character n-grams capture sub-word shape and resist obfuscation. I classify with Multinomial NB,

Complement NB [Rennie et al., 2003] (designed for imbalance), and **Logistic Regression**. A linear model over TF-IDF keeps the pipeline *interpretable* - a token’s contribution is exactly its coefficient times its TF-IDF value, and these sum to the log-odds - giving faithful model-level (global weights) and instance-level (per-token) explanations without a LIME/SHAP surrogate. Model selection uses validation F1; the test set is touched once.

Evaluation. I report accuracy but do not trust it alone: under a legit-majority prior a doing-nothing model scores high while catching no phishing (the *accuracy paradox*). Because a missed phishing (false negative) is far costlier than a false alarm (false positive), I emphasise *recall*, the recall-weighted F_2 , and **MCC**, and pick a cost-optimal threshold. In plain terms: **F1** is the harmonic mean of precision and recall (one number balancing false alarms against misses); **MCC** is a correlation over all four confusion cells that stays honest under imbalance (+1 perfect, 0 random guessing, −1 always wrong); and **AUC** is the probability the model scores a random phishing email above a random legitimate one - how well it separates the classes across every threshold. Every headline metric carries a bootstrap 95% CI (1000 resamples); classifiers are compared with McNemar’s exact test. This split-and-leakage discipline - stratified 60/20/20, dedupe before split, vectoriser fit on training data only, no email from the future - is applied throughout.

Threat model and defences. The attacker edits the email *text* but not the pipeline (test-time *evasion*), perturbing only phishing emails, so I measure **recall on phishing**. I distinguish three attack families, which is what makes the defence evaluation honest: *seen* (homoglyphs Latin→Cyrillic and leetspeak, the family the normaliser is built around), *held-out* (full-width characters, a different family), and *adaptive* (an attacker who knows about normalisation and uses confusable letters deliberately absent from the table). In the standard taxonomy these are evasion attacks; the importance-guided variant (Section 6) is *targeted grey-box*, replaying against DistilBERT tests *transferability*, and the user-report channel (Section 9) is the one genuine *poisoning* surface. I evaluate the two textbook countermeasures: **normalisation** (clean the input - map Unicode confusables back to Latin, generic NFKD fold, strip invisibles, de-leet) and **adversarial training** (augment the training set with perturbed phishing and retrain).

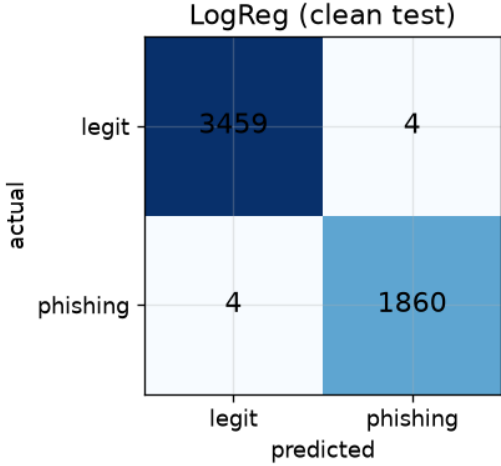
5 Clean Detection and Exploratory Analysis

Step 2 of the pipeline: build a strong, explainable baseline. On clean text the task is easy. Two dumb baselines set the scale: a *majority* classifier scores 65% accuracy while catching *zero* phishing (the accuracy paradox in one line), and a keyword list does little better (F1 0.25). Among real models (Table 1), Logistic Regression on **union** features clearly wins (F1 0.998, MCC 0.997); McNemar’s test confirms its edge over Complement NB is significant ($p \ll 0.001$), and CNB beats MNB, consistent with Rennie et al. [2003]. The confusion matrix (Fig. 1a) shows only **4 false positives** and **4 false negatives** in 5,327 emails.

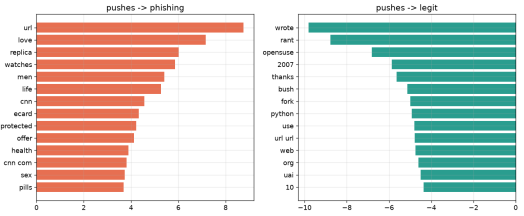
A short EDA on interpretable meta-features (length, word/digit/uppercase ratios) turns each downstream choice into a tested statement. The ratio features are strongly right-skewed (avg. word-length skewness ≈ 6.6), so I use **Spearman** rank correlation and the median. The clearest signal is length - phishing emails are *shorter* ($\rho \approx -0.57$; phishing median ≈ 290 chars vs. 1,185 for legitimate), which a non-parametric Mann–Whitney U test confirms at $p \ll 0.001$. Length and word count are collinear (variance inflation factor ≈ 21 , well past the threshold of 10), so the meta-feature block is kept **diagnostic-only** and never enters the classifier: even the strongest

Table 1: Model comparison on union features (clean test set). F1 with a bootstrap 95% CI.

Model	Accuracy	Precision	Recall	F1	F2	MCC	F1 (95% CI)
Multinomial NB	0.9807	0.9977	0.9469	0.9716	0.9566	0.9577	0.972 [0.966, 0.977]
Complement NB	0.9835	0.9972	0.9555	0.9759	0.9635	0.9638	0.976 [0.971, 0.981]
Logistic Reg.	0.9985	0.9979	0.9979	0.9979	0.9979	0.9967	0.998 [0.996, 0.999]
DistilBERT	0.9887	—	—	0.984	—	0.975	—



(a) Confusion matrix (clean test set): 4 false positives, 4 false negatives.



(b) Top signed word weights - a faithful, model-level explanation.

Figure 1: Clean-data results and interpretability.

meta-feature is only a moderate correlation, confirming the real signal is *which words appear*. (A data-quality check found 0 of 34,342 dates hit the Unix-epoch “1970” trap.)

6 Adversarial Robustness - the Main Result

Steps 3–6: attack, fix, attack the fix, and strengthen the defence. Table 2 and Figure 2 are the centre of the project. Under an **imperceptible homoglyph+leet attack** the **word-only** model collapses: recall falls from 0.99 to **0.71** - nearly 30% of phishing sails through with no visible change to a reader. Two defences answer it. Character n-grams are robust in their own right (the **union** model only drops to 0.93, because sub-word chunks survive letter swaps), and **normalisation** restores the word model to 0.99. The honest part is generalisation: against a *held-out* full-width attack the undefended word model is just as fragile (**0.68**), yet normalisation still repairs it to 0.99 - not because I listed full-width characters, but because the defence’s *generic* Unicode fold handles them. Adversarial training is the mirror image: near-perfect on the family it saw (1.00 on homoglyph+leet) but weak on the held-out family (**0.78**), because it memorised particular characters rather than the idea of obfuscation. The two therefore cover each other’s blind spots, and combining them gives ≥ 0.98 across every family - so a learned defence belongs *alongside* a principled input defence, not instead of one. Repeating every attack over $K=10$ seeds gives standard deviations of at most a few thousandths, so the collapse-and-recovery is a genuine effect; normalisation’s only cost is a small precision hit (union false-positive rate $0.0012 \rightarrow 0.0040$).

Table 2: Recall on phishing under attack and defence (test set). The word-only model collapses and is repaired; the adaptive attacker sets the ceiling.

Setting	word-only recall	union recall
clean	0.992	0.998
homoglyph+leet (seen)	0.711	0.933
+ normalise	0.992	0.998
full-width (held-out)	0.677	0.920
+ normalise	0.993	0.998
adaptive (unmapped)	0.888	0.983
+ normalise	0.963	0.987

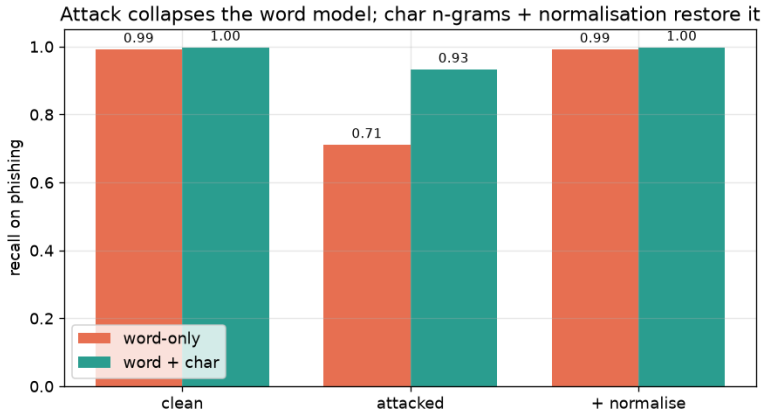


Figure 2: Adversarial robustness: word-only features are fragile; character n-grams and normalisation restore recall, up to the adaptive-attacker ceiling.

A stronger, importance-guided attacker, and a stronger defence. A real adversary spends its budget where it matters - on the few tokens the model weights most [Gao et al., 2018, Li et al., 2019]. Because my model is linear, each token’s pull is its coefficient, so I homoglyph only the top- k words. Eight *targeted* edits cut word-only recall to **0.74** (vs. 0.98 for eight *random* edits), a median of just 9 words flips a phishing email, and this *targeted* attacker lowers the fully-defended union ceiling to **0.88** - the strongest form of “no silver bullet”. The principled fix is not to keep adding table rows by hand but to fold every character through the **full Unicode confusables set (UTS #39)**, the same data browsers use for spoofed domains. Against the targeted attacker this restores recall to 0.998, because the “unmapped” glyphs it relied on *are* in the official set. This is a big, principled improvement rather than a hand-patched table: cleaning against the official confusables set restores the original text for almost all phishing, so the same obfuscation is much harder to land - where the old hand-list left a gap the adaptive attacker walked straight around - and it is nearly free (it only ever remaps non-ASCII, so ordinary ASCII mail passes untouched). The improvement covers the *character-confusable* family only; the arms race ends not in victory but by forcing the adversary onto costlier ground.

Transferability. A lexical attack replayed against DistilBERT does *not* transfer: its recall stays at 0.999, because its sub-word (WordPiece) tokeniser breaks obfuscated words into pieces much as my character n-grams do. On clean text DistilBERT (F1 0.984) slightly trails the TF-IDF union

Table 3: In-corpus vs. cross-corpus (out-of-distribution) performance.

	Accuracy	Precision	Recall	F1	MCC	AUC
In-corpus (CEAS test)	0.9985	0.9979	0.9979	0.9979	0.9967	1.000
Cross-corpus (OOD)	0.586	0.773	0.243	0.369	0.236	0.784

model, because bag-of-words already captures “which words appear”; its value here is built-in sub-word robustness, not higher accuracy.

7 Reality Checks: Space and Time

Step 7: does any of this survive outside the lab? The clean numbers are all *in-corpus*. Run the CEAS-trained model on entirely different corpora (Table 3) and F1 falls from 0.998 to **0.37**: this is distribution shift, and it is the number to quote about the real world. But I diagnose rather than despair. The model is under-confident, not confidently wrong: the OOD **AUC is still** 0.78, and lowering the threshold to 0.1 lifts F1 from 0.37 to 0.75 - a prior/threshold shift. What remains is a genuine **vocabulary gap** (out-of-vocabulary rate $0.07 \rightarrow 0.20$) that no threshold closes. In *time*, the sharpest test: on 1,303 real **2023–2025** phishing emails, a 2008-trained model catches only 14% - genuine *concept drift*. A 2008 model is Taleb’s turkey: fed every day and completely confident, right up until Thanksgiving - the past stops predicting the future. And the threat is real, not a straw man: mixed-script (homoglyph) tokens and zero-width characters are essentially absent in 2008 phishing ($\approx 0\%$) but appear in 1.8% and 4.0% of modern phishing - the very obfuscations I defend against are an emerging, non-zero technique, so a filter that ignores them is one cheap edit from evasion.

8 The Next Attack Class: Words, not Letters

Steps 9–10: with letters closed, the attacker moves to words - and I answer with a model that resists it by design. A rational attacker edits *words*. I try three word-level attacks on the phishing test emails (Table 4). Synonym substitution barely dents the model (0.998; the redundant lexical and character features absorb small changes), deletion hurts moderately, but **dilution** - scattering legitimate-looking words so their combined weight outvotes the phishing ones - is dangerous: enough padding drives recall to **0.13**, because a linear bag-of-words simply sums per-word evidence. Critically, the skeleton canonicaliser does **nothing** here ($0.998 \rightarrow 0.997$): it is the wrong defence for the wrong attack class. Adversarial training on the word attacks restores dilution recall from **0.28** to 1.00 with no clean loss, but that must be told the attack exists.

A monotone model that resists dilution by design. Can I *train* a model that resists dilution by design? Constrain every weight to be non-negative, $w_j \geq 0$, over **binary word-presence** features, so the score is $s(x) = b + \sum_j w_j \mathbf{1}[\text{word}_j \in x]$. Inserting words can only flip indicators $0 \rightarrow 1$, and every weight is non-negative, so s is **monotonically non-decreasing under insertion**: a flagged email cannot be un-flagged by adding text. This covers a different class from the UTS #39 cleaner (substitution of characters vs. insertion of words), and follows Fleshman et al. [2018]. It holds *only* for unigram features - a bigram can be broken by an inserted word - so the model is deliberately order-free. In practice, padding with +40/ +100/ +300 words lowered the score on no emails and left every clean-flagged phishing still flagged. The cost is real and stated plainly

Table 4: Union recall under word-level attacks, by budget (words changed/inserted).

budget	synonym	delete	dilution
5	0.998	0.959	0.970
10	0.998	0.912	0.638
20	0.998	0.809	0.284
40	0.998	0.781	0.129

Table 5: What the monotone model costs and buys. Recall on phishing under dilution: the union model collapses, the monotone model is flat.

Model	F1	MCC	recall	prec.
union LR	0.998	0.997	0.998	0.998
monotone ($w \geq 0$)	0.912	0.869	0.873	0.953
	+0	+10	+20	+40
union LR (diluted)	0.998	0.632	0.365	0.129
monotone (diluted)	0.869	0.869	0.869	0.866

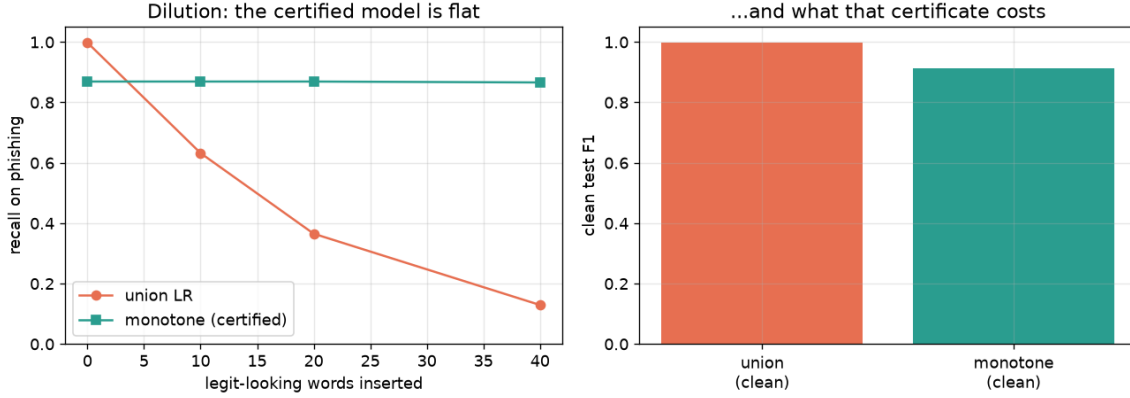


Figure 3: Left: recall as benign filler is added - the union model collapses $0.998 \rightarrow 0.129$; the monotone model is flat, because no padding can lower its score. Right: the price - clean test F1 drops $0.998 \rightarrow 0.912$.

(Table 5): clean F1 falls $0.998 \rightarrow \mathbf{0.912}$, because a guilt-only model can never use “this word looks legitimate” to acquit a borderline email. It is therefore a *safety net* run beside the main detector (flag if *either* fires), not a replacement - and it does not cover *deletion* or semantic rewriting. The character front is now hardened by the UTS #39 cleaner and word insertion can no longer clear a flag, leaving the adversary the costlier options of deleting or rewriting the scam itself (Fig. 3).

9 Keeping It Working, and Deployment

Step 8: evolve so the detector keeps working as phishing changes. Evolving on modern mail. The practical fix for drift is to feed the model modern examples. Adding modern phishing to training while leaving the legitimate-mail source unchanged (no new confound) lifts recall on held-out modern phishing from **0.147** to 0.980, *without* forgetting the 2008 scams (0.998) and *without* raising false alarms (FPR 0.007). A fully modern detector scores even higher (F1 0.996) but a source-separability probe shows its two sources are 99% distinguishable by provenance alone - the exact confound the project avoids - so I quote the confound-free number and flag the other as inflated.

Human-in-the-loop. Targeted phishing is delivered to many employees at once. Clustering the modern phishing yields 93 near-duplicate campaigns; revealing one member as “reported” and propagating by text similarity lifts recall on the rest from **0.149** to **1.000** with no extra false alarms. Recall alone proves little (any modern example helps), so the decisive figure is the

campaign-blind ablation: remove that campaign’s own report and recall falls to **0.329** - the gap 1.000 vs. **0.329** is the genuine campaign effect. Two failure modes matter: character obfuscation *defeats the mechanism completely* (homoglyphs destroy the shared tokens the similarity uses) until the UTS #39 canonicaliser is applied, so the normaliser is a *precondition* for the feature; and false reports are a poisoning channel that slowly raises the false-positive rate. A short **paraphrase** probe (an LLM rewrites the scam, a second LLM validates it is still phishing; 24 of 30 valid) leaves the union model’s recall at 1.0 - the redundant features absorb rewording - while the monotone model gives a little ground, since rewriting removes the positive evidence it exclusively relies on.

Deployment envelope. Measured, not guessed: a median scoring latency of ≈ 3 ms, an on-disk footprint of ≈ 3 MB, several hundred emails/sec/core, and a full retrain in seconds. The model is small enough to run *client-side* - inside a mail client, so content never leaves the device - and it is deliberately *one layer of many*, ignoring sender and links (the partner layer’s job). Not measured: MIME/HTML parsing, attachments, and non-English mail - this is a costed design proposal, not a deployment result.

10 Discussion

Mitigation versus a built-in defence. Most of my defences are *empirical* - they work against the attacks I ran, and their ceiling is found by attacking harder. The UTS #39 canonicaliser is a strong, principled input defence: swapping my partial hand-list for the official confusables set greatly improves robustness to the character-obfuscation family, and it is nearly free (it only remaps non-ASCII). The monotone model goes one step further: it is built so that word insertion cannot lower its score, but it costs ≈ 9 points of clean F1, because forbidding negative evidence removes information the detector was using. A built-in defence is worth having only when its price is one I can afford - which is why the monotone model is a safety net beside the main detector, not a replacement.

Complementary defences and the base rate. Adversarial training is strong on the family it saw but brittle to unseen ones; normalisation generalises but is weaker against a defence-aware attacker; stacked, they are robust across every family. A subtle point: the attack only *looks* dangerous under a realistic legit-majority prior - on a phishing-majority corpus a model that loses its features falls back to the majority class, which is “phishing”, so robustness numbers are meaningless without a realistic base rate.

Limitations. (1) CEAS_08 is from 2008; the cross-corpus collapse and the 14% modern-phishing recall show a single-corpus model does not transfer in space or time - retraining is a requirement. (2) The monotone model covers *insertion* only, at a real accuracy cost, so a structure-aware model that resists dilution *without* surrendering negative evidence remains future work, as does an open-ended semantic/paraphrase attacker. (3) DistilBERT is evaluated on the full test set but trained on a CPU-bounded subset, so its comparison is indicative. (4) I attacked only phishing emails; a full study would also consider making legitimate mail look like phishing.

Course relevance. The course gave a general project pipeline - data & EDA $\rightarrow$ clustering $\rightarrow$ temporal analysis $\rightarrow$ NLP $\rightarrow$ models $\rightarrow$ improvement - and this project is a deep dive into the last three (NLP, models, improvement) while touching every earlier stage: I ran the EDA statistics, treated near-duplicate phishing campaigns as *clusters* (Section 9), and studied *temporal* drift over

fifteen years (Section 7); the one stage I deliberately leave to the partner is fusing separate components. Concretely it instantiates the syllabus end-to-end: EDA statistics (Spearman, skew/kurtosis, Mann–Whitney, VIF), metrics and the accuracy paradox, leakage discipline, the adversarial-ML taxonomy (evasion / poisoning / transfer / targeted vs. indiscriminate), normalisation as the textbook countermeasure, and three levels of explainability - with the robustness-vs-interpretability trade-off as an explicit research question (the readable signal lives in the word features; the robust character n-grams are not individually meaningful).

11 Conclusion

I built the content layer of a fused phishing detector and used it to study robustness honestly. On clean single-corpus data it is near-perfect and explainable. Under imperceptible homoglyph obfuscation a word-only model loses a third of its recall; character n-grams and normalisation restore it and even generalise to an unseen attack; an importance-guided attacker opens a residual gap that folding through the full Unicode confusables set then strongly reduces, restoring recall. A cross-corpus test and a direct test on real 2023–2025 phishing (caught only 14%) show the laboratory numbers survive neither a change of corpus nor time - while that same modern phishing confirms the obfuscations I defend against are real and emerging - and *evolving* on modern mail lifts recall to 0.98. A word-level *dilution* attack still evades the model, answered by a monotone model whose weights only add suspicion, so inserted text cannot un-flag a scam, at an honest cost of nine points of clean F1. Word insertion can no longer clear a flag and the character front is hardened with the official confusables set, and a single simulated user report per campaign recovers most of what drift takes away - but only once the input is canonicalised, and only if the report channel is defended. The recurring lesson is that every component I add, including the ones meant to help, becomes something an attacker can aim at. Together these results argue for exactly the design this layer is built for: combine a lexical content detector with an independent non-text signal, so that when an attacker defeats one channel the other still fires.

Executive Summary

Problem. Phishing email is a leading initial-access vector and much of the deception is linguistic, so I build the *content* layer of a fused, two-part detector: read subject + body and decide phishing vs. legitimate, with explainability and - the main focus - robustness to text-obfuscation evasion. **Data.** CEAS_08 is the one large corpus with both classes from a single source, so a model cannot cheat on provenance (a corpus-ID probe reaches $\approx 99\%$); URLs stripped, rebalanced to a realistic legit-majority prior; frozen and hashed. **Method.** TF-IDF (word + char n-grams) $\rightarrow$ Naive Bayes / Logistic Regression, tuned on validation, evaluated once with bootstrap CIs and McNemar; homoglyph/leet attacks defended by normalisation and adversarial training against *seen*, *held-out* and *adaptive* attacks; DistilBERT baseline. **Findings.** Clean, in-corpus: near-perfect (F1 0.998, MCC 0.997), interpretable. A word-only model is fragile - an imperceptible homoglyph attack drops recall $0.99 \rightarrow \mathbf{0.71}$; char n-grams + normalisation restore it and generalise to an unseen full-width attack. An importance-guided attacker lowers the ceiling, but a UTS #39 canonicaliser (the official confusables list) restores recall and makes the attack much harder to land. Cross-corpus F1 falls to **0.37** (diagnosed as prior-shift + vocabulary gap), and a 2008 model catches **14%** of real 2023–2025 phishing - in which, unlike 2008, homoglyph/zero-width obfuscation actually appears. A word-level dilution attack evades the model ($\rightarrow \mathbf{0.13}$) but adversarial training repairs it, and *evolving* on modern mail lifts recall **0.15** $\rightarrow$ 0.98. Dilution is then answered by a monotone model over binary

word-presence whose weights only add suspicion, so inserted text cannot un-flag a scam, at a cost of clean F1 $0.998 \rightarrow \mathbf{0.912}$ - a safety net, not a replacement. **Relation to prior claims.** Two published claims tested and *held*: Complement NB’s advantage under imbalance [Rennie et al., 2003], and the collapse-then-shield behaviour under visual attack [Eger et al., 2019]. **Explored ideas.** Beyond the core detector I also sketch several forward-looking directions: an LLM-paraphrase attack (one LLM rewrites the scam, a second judges it is still phishing), a hand-crafted sentiment/urgency feature, edit-distance (Levenshtein) matching against known brand words, a self-updating attack database, and - the whole point - fusing this content channel with the independent URL/sender detector. **Recommendation.** A lexical content detector is an excellent, cheap, interpretable first line but brittle alone - to obfuscation and to distribution shift - which is the case for the fused design (content + an independent non-text signal), pairing a principled input defence with adversarial training.

References

- Nicholas Boucher, Ilia Shumailov, Ross Anderson, and Nicolas Papernot. Bad characters: Imperceptible NLP attacks. In *IEEE Symposium on Security and Privacy (S&P)*, pages 1987–2004, 2022.
- William B Cavnar and John M Trenkle. N-gram-based text categorization. In *Proceedings of SDAIR-94, 3rd Annual Symposium on Document Analysis and Information Retrieval*, pages 161–175, 1994.
- Davide Chicco and Giuseppe Jurman. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21(1):6, 2020.
- Steffen Eger, Gözde Gül Şahin, Andreas Rücklé, Ji-Ung Lee, Claudia Schulz, Mohsen Mesgar, Krishnkant Swarnkar, Edwin Simpson, and Iryna Gurevych. Text processing like humans do: Visually attacking and shielding NLP systems. In *Proceedings of NAACL-HLT*, pages 1634–1647, 2019.
- William Fleshman, Edward Raff, Jared Sylvester, Steven Forsyth, and Mark McLean. Non-negative networks against adversarial attacks. *arXiv preprint arXiv:1806.06108*, 2018.
- Ji Gao, Jack Lanchantin, Mary Lou Soffa, and Yanjun Qi. Black-box generation of adversarial text sequences to evade deep learning classifiers. In *IEEE Security and Privacy Workshops (SPW)*, pages 50–56, 2018.
- Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*, 2015.
- Jinfeng Li, Shouling Ji, Tianyu Du, Bo Li, and Ting Wang. TextBugger: Generating adversarial text against real-world applications. In *Network and Distributed System Security Symposium (NDSS)*, 2019.
- Daniel Lowd and Christopher Meek. Good word attacks on statistical spam filters. In *Proceedings of the Second Conference on Email and Anti-Spam (CEAS)*, 2005.

- Danish Pruthi, Bhuwan Dhingra, and Zachary C Lipton. Combating adversarial misspellings with robust word recognition. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 5582–5591, 2019.
- Jason D Rennie, Lawrence Shih, Jaime Teevan, and David R Karger. Tackling the poor assumptions of Naive Bayes text classifiers. In *Proceedings of the 20th International Conference on Machine Learning (ICML)*, pages 616–623, 2003.
- Mehran Sahami, Susan Dumais, David Heckerman, and Eric Horvitz. A Bayesian approach to filtering junk e-mail. In *AAAI Workshop on Learning for Text Categorization*, 1998.
- Said Salloum, Tarek Gaber, Sunil Vadera, and Khaled Shaalan. A systematic literature review on phishing email detection using natural language processing techniques. *IEEE Access*, 10: 65703–65727, 2022.
- Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- The Unicode Consortium. Unicode technical standard #39: Unicode security mechanisms. Technical report, Unicode, Inc., 2023. <https://www.unicode.org/reports/tr39/>.